МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ПОВОЛЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра ИиСП

Отчет
по лабораторной работе № 1

по дисциплине «Машинно-зависимые языки программирования»
Вариант 4

Выполнил: ст. гр. ПС-13
Петялин А.Н.
Проверил:
доцент кафедры ИиСП
Баев А.А.

г. Йошкар-Ола
2025

Цель работы:

Изучить элементы и принципы процедуры дизассемблирования программ.

Задания на лабораторную работу:

Провести процедуру восстановления ассемблерного кода по hex коду.

1. **Теоретические сведения**

У нас есть hex код:

:100000000C9434000C943E000C943E000C943E0082

:100010000C943E000C943E000C943E000C943E0068

:100020000C943E000C943E000C943E000C943E0058

:100030000C943E000C943E000C943E000C943E0048

:100040000C943E000C943E000C943E000C943E0038

:100050000C943E000C943E000C943E000C943E0028

:100060000C943E000C943E0011241FBECFEFD8E04C

:10007000DEBFCDBF0E9440000C9452000C940000E3

:10008000539A52985A9A4A9902C05B9A01C05B9857

:1000900022E58EE497E1215080409040E1F700C0D6

:0800A00000C0F1CFF894FFCF7E

:00000001FF

Нам нужно восстановить ассемблерный код (произвести дизассемблирование).Разберем для примера первую строку.

:100000000C9434000C943E000C943E000C943E0082

Разложим эту строку таким образом:

:10 0000 00 0C94 3400 0C94 3E00 0C94 3E00 0C94 3E00 82

:LL AAAA TT DDDD DDDD DDDD DDDD DDDD DDDD DDDD DDDD CC

Расшифровка полей записи:

- Каждая запись в файле Intel HEX должна начинаться с двоеточия.
- LL - Поле длины — показывает количество байт данных (DD) в записи.
- AAAA - Поле адреса — представляет начальный адрес записи.
- TT - Поле типа. Оно может принимать следующие значения:
  - 00 запись содержит данные двоичного файла.
  - 01 запись является концом файла.

- 02 запись адреса сегмента (подробнее см. ниже).
- 03 Стартовый адрес запуска программы в формате 80x86 (значения регистров CS:IP счетчика команд и регистра смещения)
- 04 запись расширенного адреса (подробнее см. ниже).
- 05 32-битный адрес запуска программы в «линейном» адресном пространстве

- DD - Поле данных. Запись может содержать несколько байт данных. Количество байт данных должно соответствовать полю LL.
- CC - Поле контрольной суммы.

Для перевода записи из hex-кода в ассемблерный,нам потребуется только поле DD..:

0C94 3400 0C94 3E00 0C94 3E00 0C94 3E00

Поменяем байты местами (это нужно из-за того,что в памяти информация храниться в обратном порядке (little-endian).

Получим:

940C 0034 940C 003E 940C 003E 940C 003E

Переведем первые четыре байта из 16-ой системы счисления в двоичную.

Получим:

1001 0100 0000 1100 0000 0000 0011 0100

На сайте https://www.mcu4you.ru/tablicy-komand-assemblera-avr/ сравниваем получившийся двоичный код и находим команду, под маску которой подходит наша строка.

Так как наше число начинается на 94, то подходящими командами будут две- jmp и call.Однако, сравнив маски этих команд мы видим, что нам подходит команда jmp, из-за того, что 13,14 и 15 биты в нашей строке -110, что соответствует только команде jmp.

Сравним нашу строку и маску команды jmp:

1001 0100 0000 1100 0000 0000 0011 0100

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

Найдем k

K=0 0000 0 0000 0000 0011 0100

Убрав незначащие нули, получаем что k = 11 0100.Однако,при дизассемблировании нужно для команд rjmp, jmp, call сместить адрес на один бит влево,тогда k= 1101000.

Почему необходимо выполнять смещение адреса? Смещение влево увеличиваем значение в два раза. В коде команды содержится информация о значении программного счетчика, а память команд (FLASH program Memory) у нас 16ти битная и адреса указываются соответственно.

Переведем k в шестнадцатиричную систему счисления,получаем : k=0x68.

Те же действия производим со следующими командами.

В итоге получаем строки:

0: 0c 94 34 00 jmp 0x68;

4: 0c 94 3e 00 jmp 0x7c;

8: 0c 94 3e 00 jmp 0x7c;

c: 0c 94 3e 00 jmp 0x7c;

В остальных случаях, коды команда 16-ти битные, из-за чего выполнять смещение нет необходимости.

С оставшимися строками нужно производить те же операции, но есть команды, при работе с которыми нужно совершить дополнительные действия.

Если у нас команды ldi и subi:

Регистр d начинает свой отчет от 16, следовательно значение d нужно прибавить к 16, для правильной записи команды.

Если у нас команды, требующие знака для полученного числа (например, rjmp, breq, bren), то:

1. Смотрим на первый бит маски.

2. Если первый бит – 1, значит полученное число будет иметь знак минус. Для получения самого числа, мы инверсируем строку, прибавляем единицу и делаем сдвиг на 1 бит влево. Полученное число переводим в 10-ую систему счисления и перед ним ставим знак минус.
3. Если первый бит – 0, значит полученное число будет иметь знак плюс. Для получения самого числа, делаем смещение на 1 бит влево и переводим в 10-ую систему счисления и перед ним ставим знак плюс.

Номера строк записываются в 16-ой системе счисления. Сам номер увеличивается либо на +2, либо на +4. Это зависит от команды, так например, после команд call или jmp, которые занимают 4 байта, мы прибавляем 4. После команд, которые занимают 2 байта, прибавляем к номеру строки 2.

2. Практическая часть

Результат:

0: 0c 94 34 00 jmp 0x68;

4: 0c 94 3e 00 jmp 0x7c;

8: 0c 94 3e 00 jmp 0x7c;

c: 0c 94 3e 00 jmp 0x7c;

10: 0c 94 3e 00 jmp 0x7c;

14: 0c 94 3e 00 jmp 0x7c;

18: 0c 94 3e 00 jmp 0x7c;

1c: 0c 94 3e 00 jmp 0x7c;

20: 0c 94 3e 00 jmp 0x7c;

24: 0c 94 3e 00 jmp 0x7c;

28: 0c 94 3e 00 jmp 0x7c;

2c: 0c 94 3e 00 jmp 0x7c;

30: 0c 94 3e 00 jmp 0x7c;

34: 0c 94 3e 00 jmp 0x7c;

38: 0c 94 3e 00 jmp 0x7c;

3c: 0c 94 3e 00 jmp 0x7c;

40: 0c 94 3e 00 jmp 0x7c;

44: 0c 94 3e 00 jmp 0x7c;

48: 0c 94 3e 00 jmp 0x7c;

4c: 0c 94 3e 00 jmp 0x7c;

50: 0c 94 3e 00 jmp 0x7c;

54: 0c 94 3e 00 jmp 0x7c;

58: 0c 94 3e 00 jmp 0x7c;

5c: 0c 94 3e 00 jmp 0x7c;

60: 0c 94 3e 00 jmp 0x7c;

64: 0c 94 3e 00 jmp 0x7c;

68: 11 24 eor r1, r1;

6a: 1f be out 0x3f, r1;

```
6c: cf ef ser r28;
6e: d8 e0 ldi r29, 0x08;
70: de bf out 0x3e, r29;
72: cd bf out 0x3d, r28;
74: 0e 94 40 00 call 0x80;
78: 0c 94 50 00 jmp 0xa0;
7c: 0c 94 00 00 jmp 0;
80: 53 9a sbi 0x0a, 3;
82: 52 98 cbi 0x0a, 2;
84: 5a 9a sbi 0x0b, 2;
86: 4a 99 sbic 0x09, 2;
88: 02 c0 rjmp .+4;
8a: 5b 9a sbi 0x0b, 3;
8c: 01 c0 rjmp .+2;
8e: 5b 98 cbi 0x0b, 3;
90: 22 e5 ldi r18, 0x52;
92: 8e e4 ldi r24, 0x4E;
94: 97 e1 ldi r25, 0x17;
96: 21 50 subi r18, 0x01;
98: 80 40 sbci r24, 0x00
9a: 90 40 sbci r25, 0x00;
9c: e1 f7 brne .-8;
9e: 00 c0 rjmp .+0;
a0: 00 c0 rjmp .+0;
a2: f1 cf rjmp .-30;
a4: f8 94 cli ;
a6: ff cf rjmp .-2;
```

Подробное дизассемблирование

:10 0000 00 0C94 3400 0C94 3E00 0C94 3E00 0C94 3E00 82

0C94 3400

940C 0034

jmp

1001 0100 0000 1100 0000 0000 0011 0100

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 0100=0011 0100

Сдвигаем влево на один бит

k=0110 1000=0x68

0: 0c 94 34 00 jmp 0x68;

Команда jmp отправит нас по адресу 0x68.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

4: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

8: 0c 94 3e 00 jmp 0x7c;

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

c: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

:10 0010 00 0C94 3E00 0C94 3E00 0C94 3E00 0C94 3E00 68

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

10: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

14: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

18: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

1c: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

:10 0020 00 0C94 3E00 0C94 3E00 0C94 3E00 0C94 3E00 58

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

20: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

24: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

28: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

2c: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

:10 0020 00 0C94 3E00 0C94 3E00 0C94 3E00 0C94 3E00 48

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

30: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

34: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

38: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

3c: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

:10 0030 00 0C94 3E00 0C94 3E00 0C94 3E00 0C94 3E00 38

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

40: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

44: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110
1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110
Сдвигаем влево на один бит
k=0111 1100=0x7C

48: 0c 94 3e 00 jmp 0x7c;
Команда jmp отправит нас по адресу 0x7C.

0C94 3E00
940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110
1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110
Сдвигаем влево на один бит
k=0111 1100=0x7C

4c: 0c 94 3e 00 jmp 0x7c;
Команда jmp отправит нас по адресу 0x7C.

:10 0040 00 0C94 3E00 0C94 3E00 0C94 3E00 0C94 3E00 28

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

50: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

54: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

58: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

5c: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

:10 0060 00 0C94 3E00 0C94 3E00 1124 1FBE CFEF D8E0 4C

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

60: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

0C94 3E00

940C 003E

jmp

1001 0100 0000 1100 0000 0000 0011 1110

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0011 1110

Сдвигаем влево на один бит

k=0111 1100=0x7C

64: 0c 94 3e 00 jmp 0x7c;

Команда jmp отправит нас по адресу 0x7C.

1124

2411

eor

0010 0100 0001 0001

0010 01rd dddd rrrr

r=0 0001

d=0 0001

68: 11 24 eor r1, r1;

Исключающее или для регистра r1, результат 0.

1FBE

BE1F

out

1011 1110 0001 1111

1011 1PPr rrrr PPPP

P=11 1111=0x3F

r=0 0001

6a: 1f be out 0x3f, r1;

Команда сохраняет данные регистра R1 в регистровом файле пространства I/O.
Из даташита выясняем имя регистра 0x3f – SREG.
Данная команда установила 1 в бите C - Флаг переноса.

CFEF

EFCF

ser

1110 1111 1100 1111

1110 1111 dddd 1111

d=1100=0xC=12

6c: cf ef ser r28;

Значение заносится непосредственно в регистр назначения R28.

D8E0

E0D8

ldi

1110 0000 1101 1000
1110 KKKK dddd KKKK

K=0000 1000=0x08
d=1101=0xD=13

6e: d8 e0 ldi r29, 0x08;
Загружается 8-разрядная константа в регистр от 16 по 31.

:10 0070 00 DEBF CDBF 0E94 4000 0C94 5200 0C94 0000 E3

DEBF
BFDE

out

1011 1111 1101 1110
1011 1PPr rrrr PPPP

P=11 1110=0x3E
r=1 1101=0x1D

70: de bf out 0x3e, r29;
Команда сохраняет данные регистра R29 в регистровом файле пространства I/O.
0x3E – регистр SPH. Stack Pointer High and Stack Pointer Low Register

CDBF

BFCD

out

1011 1111 1100 1101

1011 1PPr rrrr PPPP

P=11 1101=0x3D

r=1 1100=0x1C

72: cd bf out 0x3d, r28;

Команда сохраняет данные регистра R29 в регистровом файле пространства I/O.

0x3D – регистр SPL. Stack Pointer High and Stack Pointer Low Register

0E94 4000

940E 0040

call

1001 0100 0000 1110 0000 0000 0100 0000

1001 010k kkkk 111k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0100 0000=0100 0000

Сдвигаем влево на один бит

k=1000 0000=0x80

74: 0e 94 40 00 call 0x80;

Вызов подпрограммы из памяти программ. Адрес возврата (к команде после CALL)
сохраняется в стеке. Переход на команду по адресу 0x80

0C94 5000

940C 0050

jmp

1001 0100 0000 1100 0000 0000 0101 0000

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0101 0000=0101 0000

Сдвигаем влево на один бит

k=1010 0000=0xA0

78: 0c 94 50 00 jmp 0xa0;

Команда jmp отправит нас по адресу 0xA0.

0C94 0000

940C 0000

jmp

1001 0100 0000 1100 0000 0000 0000 0000

1001 010k kkkk 110k kkkk kkkk kkkk kkkk

k=0 0000 0 0000 0000 0000 0000=0

7c: 0c 94 00 00 jmp 0;

Команда jmp отправит нас по адресу 0x00

:10 0080 00 539A 5298 5A9A 4A99 02C0 5B9A 01C0 5B98 57

539A

9A53

sbi

1001 1010 0101 0011

1001 1010 PPPP Pbbb

P=01010=1010=0x0A

b=011=0x03

80: 53 9a sbi 0x0a, 3;

Команда устанавливает заданный бит в регистр I/O. В регистре 0x0A – DDRD устанавливает бит 3

5298

9852

cbi

1001 1000 0101 0010

1001 1000 PPPP Pbbb

P=01010=0x0A
b=010=0x02

82: 52 98 cbi 0x0a, 2;
В регистре 0x0A – DDRD очищается бит 2.

5A9A
9A5A

sbi

1001 1010 0101 1010
1001 1010 PPPP Pbbb

P=01011=0x0B
b=010=0x02

84: 5a 9a sbi 0x0b, 2;
Команда устанавливает бит 2 в регистре 0x0B-PORTD.

4A99
994A

sbic

1001 1001 0100 1010

1001 1001 PPPP Pbbb

P=01001=0x09

b=010=0x02

86: 4a 99 sbic 0x09, 2;

Команда проверяет состояние бита в регистре I/O и, если этот бит очищен, пропускает следующую команду. Данная команда работает с младшими 32 регистрами I/O (адреса с 0 по 31).

02C0

C002

rjmp

1100 0000 0000 0010

1100 kkkk kkkk kkkk

k=0000 0000 0010=0010

первый бит 0,значит число с плюсом.

сделаем сдвиг на 1 бит влево.

k=0100=+4

88: 02 c0 rjmp .+4;

Команда выполняет относительный переход по адресу 0x92.

5B9A

9A5B

sbi

1001 1010 0101 1011

1001 1010 PPPP Pbbb

P=01011=0x0B

b=011=0x03

8a: 5b 9a sbi 0x0b, 3;

Команда устанавливает бит 3 в регистре 0x0B-PORTD.

01C0

C001

rjmp

1100 0000 0000 0001

1100 kkkk kkkk kkkk

k=0000 0000 0001

первый бит 0,значит число с плюсом.

сделаем сдвиг на 1 бит влево.

k=0010=2

8c: 01 c0 rjmp .+2;

Команда выполняет относительный переход по адресу 0x92.

5B98

985B

cbi

1001 1000 0101 1011

1001 1000 PPPP Pbbb

P=01011=0x0B

b=011=0x03

8e: 5b 98 cbi 0x0b, 3;

В регистре 0x0B – PORTD очищается бит 3.

:10 0090 00 22E5 8EE4 97E1 2150 8040 9040 E1F7 00C0 D6

22E5

E522

ldi

1110 0101 0010 0010

1110 KKKK dddd KKKK

K=0101 0010=0x52

d=0010=2

90: 22 e5 ldi r18, 0x52;

Загружает значение 82 в регистр R18.

8EE4

E48E

ldi

1110 0100 1000 1110

1110 KKKK dddd KKKK

K=0100 1110=0x4E

d=1000=8

92: 8e e4 ldi r24, 0x4E;

Загружает значение 78 в регистр R24.

97E1

E197

ldi

1110 0001 1001 0111

1110 KKKK dddd KKKK

K=0001 0111=0x17

d=1001=9

94: 97 e1 ldi r25, 0x17;

Загружает значение 23 в регистр R25.

2150

5021

subi

0101 0000 0010 0001

0101 KKKK dddd KKKK

K=0000 0001=0x01

d=0010=0x02

96: 21 50 subi r18, 0x01;

Вычитание константы 1 из содержимого регистра, размещение результата в регистре назначения R18.

8040

4080

sbci

0100 0000 1000 0000

0100 KKKK dddd KKKK

K=0000 0000=0x00

d=1000=8

98: 80 40 sbci r24, 0x00;

Вычитание константы 0 и содержимого флага переноса (C) из содержимого регистра, размещение результата в регистре назначения R24.

9040

4090

sbci

0100 0000 1001 0000

0100 KKKK dddd KKKK

K=0000 0000=0x00

d=1001=9

9a: 90 40 sbci r25, 0x00;

Вычитание константы 0 и содержимого флага переноса (C) из содержимого регистра, размещение результата в регистре назначения R25.

E1F7

F7E1

brne

1111 0111 1110 0001

1111 01kk kkkk k001

k=1111100

первый бит 1,значит число будет отрицательное.

инверсируем строку,получаем k=0000011

Прибавляем единицу,k=0100

делаем сдвиг на один бит влево,в итоге k=1000=-8

9c: e1 f7 brne .-8;

Условный относительный переход. Тестируется бит флага нулевого значения (Z) регистра статуса и, если бит очищен, выполняется переход относительно состояния счетчика программ.

00C0

C000

rjmp

1100 0000 0000 0000

1100 kkkk kkkk kkkk

k=0000 0000 0000

Первый бит 0,значит число с плюсом.Сдвиг влево ничего не меняет,значит k=0

9e: 00 c0 rjmp .+0;

Команда выполняет относительный переход по адресу 0xA0.

:08 00A0 00 00C0 F1CF F894 FFCF 7E

00C0

C000

rjmp

1100 0000 0000 0000

1100 kkkk kkkk kkkk

k=0000 0000 0000

Первый бит 0,значит число с плюсом.Сдвиг влево ничего не меняет,значит k=0

a0: 00 c0 rjmp .+0;

Команда выполняет относительный переход по адресу 0xA2.

F1CF

CFF1

rjmp

1100 1111 1111 0001

1100 kkkk kkkk kkkk

k=1111 1111 0001

первый бит 1,значит число с минусом.

инвертируем строку,получаем k=0000 0000 1110

добавляем единицу,k=0000 0000 1111

делаем сдвиг на один бит влево,в итоге k=0000 0001 1110=-30

a2: f1 cf rjmp .-30;

Команда выполняет относительный переход по адресу 0x44.

F894

94F8

cli

1001 0100 1111 1000

a4: f8 94 cli ;

Команда запрещает прыревания(очищает флаг глобального прерывания I в регистре статуса SREG).

FFCF

CFFF

rjmp

1100 1111 1111 1111

1100 kkkk kkkk kkkk

k=1111 1111 1111

первый бит 1,значит число с минусом.

инвертируем строку,получаем k=0000 0000 0000

добавляем единицу,k=0000 0000 0001

делаем сдвиг на один бит влево,в итоге k=0000 0000 0010=-2

a6: ff cf rjmp .-2;

Команда выполняет относительный переход по адресу 0xA4.

:00000001FF

Это служебная строка конца файла в формате Intel HEX.

Выводы

1. Я научился получать нужную информацию из hex-кодов.
2. Я научился восстанавливать ассемблерный код из hex-кодов.
3. Я изучил, что делают различные команды и узнал о некоторых регистрах.